DualClusterNet: Personalized Pattern-Space Composition for Next-POI Recommendation

Ziyang Su
Shanghai Jiao Tong University
Shanghai, China
suziyang@sjtu.edu.cn

Guanjie Zheng✉
Shanghai Jiao Tong University
Shanghai, China
gjzheng@sjtu.edu.cn

Jilin Hu
East China Normal University
Shanghai, China
hujilin1229@gmail.com

Linghe Kong
Shanghai Jiao Tong University
Shanghai, China
linghe.kong@sjtu.edu.cn

Abstract

Next point-of-interest (POI) recommendation supports personalized place discovery on location-based Web platforms. Mobility routines can recur across users while being realized at different places and adapted to individual preferences. We present DualClusterNet, a causal sequential recommender that composes two population-shared prototype dictionaries. At each observed prefix, soft routers retrieve pattern and spatial mixtures; a factorized multiplicative interaction couples them, feature-wise affine modulation adapts the shared representation, and a separate user-conditioned branch with an additive–projective gate retains individual variation. The prediction head combines this representation with a learned first-order transition term and ranks the complete training POI vocabulary. We evaluate on NYC and TKY using a training-only vocabulary and an original-final-target protocol that neither bridges unknown POIs nor substitutes earlier targets. The available single-seed results show competitive full-candidate ranking, while controlled analyses support the contribution of the compositional and personalized branches. Code is available at https://anonymous.4open.science/r/DualClusterNet-F4EE/.

CCS Concepts

• **Information systems** → **Recommender systems**; *Location based services.*

Keywords

Next-POI recommendation, mobility patterns, spatial context, soft clustering, personalized composition

ACM Reference Format:
Ziyang Su, Guanjie Zheng, Jilin Hu, and Linghe Kong. 2027. DualClusterNet: Personalized Pattern-Space Composition for Next-POI Recommendation. In *Proceedings of The ACM Web Conference 2027 (WWW '27).* ACM, New York, NY, USA, 10 pages.

1 Introduction

Location-based Web platforms connect online place discovery with users' physical activities. Their ranking systems must select useful destinations from a city-wide catalog while accounting for the order, timing, and geography of past visits. Next point-of-interest (POI) recommendation formalizes one part of this problem: given an observed check-in history, rank the concrete POI that the user will visit next [5, 13]. This work addresses the *Search, Recommendation, and Retrieval-Augmented AI* setting through a specific Web recommendation challenge: reusing behavioral regularities across users whose spatial contexts and individual preferences differ. The prediction concerns the immediate next place, not a route, a new-to-the-user destination, or a POI conditional on a supplied future timestamp.

A useful distinction is between a routine and its realization. Consider an illustrative routine such as visiting a workplace, obtaining a meal, and then stopping at a shop. Two users may exhibit a similar ordering of activities without sharing a single POI. Conversely, the same neighborhood may host different routines at different times, and two users in the same neighborhood need not prefer the same restaurant. A model should therefore have a way to reuse population-level regularities, condition them on where a trajectory is unfolding, and retain user-specific variation. The example motivates an inductive bias; it does not presume that latent clusters are already validated semantic labels such as "commuting" or "shopping."

Existing methods provide complementary ingredients. Recurrent and attention-based models encode temporal and spatial dependencies [3, 5, 14]. Long- and short-term preference methods distinguish persistent tastes from recent behavior [10]. Graph and hypergraph approaches exploit transitions and collaborative relations across users [2, 11, 13]. Recent work also explicitly models latent preferences or personalized spatio-temporal clusters [15]. Consequently, neither sharing information across users nor introducing clusters is, by itself, the distinction of this work. Our focus is the *composition rule*: how two reusable dictionaries interact at each trajectory prefix, and how their joint representation is adapted to a user before ranking places.

This focus yields three design questions. First, a trajectory may move between routines rather than belonging permanently to one cluster. The assignment should therefore depend on its current prefix and remain soft. Second, spatial context should affect the expression of a pattern, rather than simply contribute an unrelated feature to a concatenation. Third, shared structure should not force users to have identical representations. An additive residual and a user-conditioned interaction offer different ways to preserve individuality, and their relative influence can depend on the current state.

We instantiate these ideas in DualClusterNet. A causal LSTM encodes the observed POI, category, temporal, geographic, and user context. Two assignment networks retrieve mixtures from a pattern dictionary and a spatial dictionary. An elementwise product of their projected mixtures implements a factorized interaction over all pattern–space pairs. Feature-wise affine modulation then adapts this shared interaction to the user. A separate user-conditioned branch captures information not constrained to pass through the

WWW '27, Dublin, Ireland
2027.

dictionaries. Finally, a scalar gate mixes additive fusion with an affine transform of the branches' multiplicative interaction. A full-candidate score combines the resulting preference representation with a learned first-order POI transition term.

Evaluation must preserve what "next" means. If an unseen POI interrupts a trajectory, deleting it and joining the remaining visits introduces a transition that was not observed. If the final POI is unknown, replacing it with an earlier known POI changes the prediction target. We instead use a training-only candidate vocabulary and retain only the final continuous known suffix for each held-out trajectory. All compared implementations receive the same valid queries, candidate set, ranking metrics, and validation-based checkpoint selection rule. This defines a closed-set experiment; it is not an evaluation of unseen-user or unseen-POI generalization.

The contributions of this study are as follows.

- We introduce two shared dictionaries whose assignments adapt to the observed prefix, together with a pattern–space interaction and separate personalized branches.
- We give an explicit factorization of its pairwise interaction and an additive–projective fusion rule, making the roles and limits of the shared and personalized components precise.
- We provide a unified-protocol comparison with nine local baseline implementations on two cities. The current single-seed results lead NYC Acc@1 and MRR and remain competitive on TKY; they do not establish uniform superiority across metrics or cities.

2 Related Work

2.1 Sequential and Context-Aware POI Recommendation

Sequential recommendation combines personal preference with dependence on recent observations. FPMC factorizes personalized Markov transitions [8]; we adapt its singleton-target form for next-POI scoring. ST-RNN adds spatial and temporal context to recurrent transitions [3]. STGN and its coupled-gate STGCN variant control recurrent updates with spatio-temporal inputs [14]. PLSPL combines long- and short-term preferences [10]. STAN uses spatio-temporal attention beyond adjacent check-ins [5]. DeepMove attends over historical trajectories to capture mobility periodicity [1], while Flashback retrieves past recurrent states according to spatio-temporal similarity [12]. LSTPM pairs a nonlocal long-term preference module with a geography-aware short-term state [9]. Together, these methods show that useful context may come from distant history as well as the latest transition.

Historical attention and prototype composition represent different sources of reuse. Attention and state retrieval reweight representations tied to recorded visits, whereas our dictionaries are population-shared parameters that can be mixed even when users do not share POIs. Likewise, long- and short-term modules separate temporal horizons, while DualClusterNet separates pattern and spatial axes and lets the spatial mixture modulate the pattern mixture. It uses a single causal recurrent encoder, does not require a supplied next-visit timestamp, and ranks the complete candidate vocabulary. This distinction motivates evaluating the interaction itself rather than treating memory, attention, and clustering as interchangeable mechanisms.

2.2 Collaborative, Graph, and Hypergraph Models

GETNext adds a global trajectory flow map to a Transformer [13]. STHGCN captures higher-order trajectory relations with spatio-temporal hypergraphs [11]. DCHL learns disentangled collaborative, transition, and geographic views [2]. HVGAE studies hyperbolic graph representations and variational next-POI modeling [4]. These methods expose population-level information beyond one user's sequence, but construct their relations and representations differently.

Our model does not perform graph message passing or hypergraph convolution. It shares prototype parameters across the training population and uses a prefix-conditioned combination of them. This is a different architectural bias, not a claim that dictionary sharing is a substitute for every form of collaborative graph information. In particular, the dictionaries are learned jointly with the recommendation objective and are not extracted as communities from a user–POI graph.

2.3 Latent Preferences, Clustering, and Personalization

Clustering and multi-interest modeling are closely related to our motivation. CLLP models latent preferences using preference prototypes, distillation, and contrastive learning [15]. DPRL investigates disentangled and personalized representations for next-POI recommendation [7]. These are important neighboring approaches: the use of a latent pattern bank or personalized spatial information is already established, so a novelty claim must concern the particular interaction and conditioning mechanism rather than clustering alone.

DualClusterNet separates two dictionary axes and combines their mixtures multiplicatively before user-specific modulation. The modulation follows the feature-wise affine conditioning principle of FiLM [6], with sigmoid-constrained scales in our implementation. A second user-conditioned branch and a learned gate distinguish additive correction from multiplicative interaction. Unlike objectives designed to enforce independence between latent factors, our next-POI loss contains no decorrelation, contrastive, or cluster-balancing term. Accordingly, "shared" denotes shared parameters and reusable components, not an identified user-invariant subspace. Section 4.4 tests the predictive contribution of the composition and personalization components; semantic interpretability of the learned factors remains an open question.

3 Methodology

Figure 1 summarizes the DualClusterNet computation. The architecture comprises a causal context encoder, pattern–space composition, user-specific adaptation, and a full-candidate prediction head.

3.1 Problem Formulation and Information Boundary

Let $\mathcal{U}$, $\mathcal{V}$, and $\mathcal{C}$ denote the user, POI, and category vocabularies observed in the training split. Each POI $v \in \mathcal{V}$ has training-derived coordinates ℓ_v and category c_v. A user's observed trajectory prefix is $H_{u,t} = (x_1, \ldots, x_t)$, where $x_j = (v_j, \tau_j)$ records a POI and timestamp.

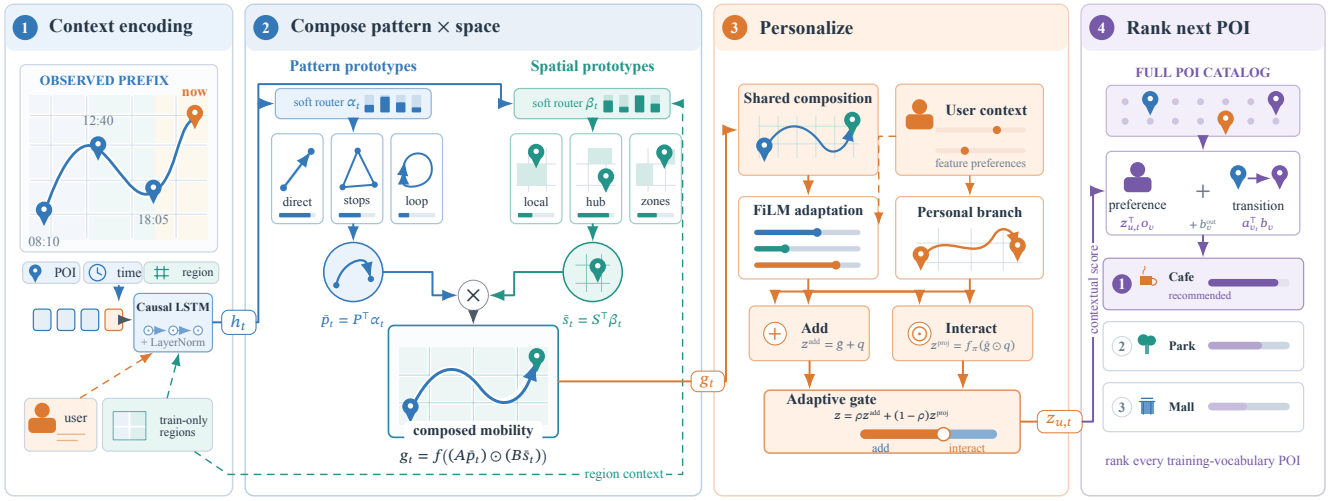


Figure 1: Overview of DualClusterNet at an observed trajectory prefix. (1) A causal encoder uses only observed check-ins and training-derived context. (2) Prefix-dependent soft routers mix population-shared pattern and spatial prototypes; their projected mixtures interact multiplicatively to express a mobility pattern in its current spatial context. (3) User FiLM and a separate personal branch feed additive and projective paths, which an adaptive scalar gate interpolates. (4) Contextual preference, the transition from the latest observed POI, and candidate bias rank the full training POI vocabulary. Prototype-card labels are schematic motifs rather than learned semantic assignments. Training-only geographic regions are context features and are distinct from the learned spatial prototypes.

The task is to rank every candidate $v \in \mathcal{V}$ according to a score $s(v \mid u, H_{u,t})$, with the actual next POI v_{t+1} as the single target. Repeated visits are valid targets. The future timestamp τ_{t+1}, future location, and future category are not query inputs.

We learn parameters from training trajectories and select a checkpoint on validation trajectories. For held-out evaluation, an unknown POI terminates the current known segment. Only the continuous known suffix ending at the original final check-in is eligible, and it must contain at least one historical check-in plus its target. An unknown user or final target makes the trajectory ineligible. This policy is formalized in Section 4.1. It preserves the original final prediction question rather than inventing a new transition after filtering.

We use d_e for the POI embedding dimension, d_c for each context embedding dimension, and d for the hidden dimension. The number of geographic partitions is R, the number of pattern prototypes is K_p, and the number of latent spatial prototypes is K_s. These three counts need not coincide.

3.2 Geographic Context and Causal Trajectory Encoding

Training-only geographic partitions. We standardize the latitude and longitude of training POIs using training-coordinate means and scales, then fit KMeans to obtain $r(v) \in \{1, \ldots, R\}$. A learned region embedding $\boldsymbol{e}^r_{r(v)} \in \mathbb{R}^{d_c}$ provides coarse geographic context. The partitioning is fixed before neural training. These coordinate clusters are proxies for local activity areas; they are not externally annotated business districts, nor are they the latent spatial prototypes introduced below.

Observed context. For check-in t, let a_t and w_t be its hour of day and day of week. We compute elapsed time $\Delta\tau_t$ and geographic distance $\Delta\ell_t$ from the preceding *observed* check-in in the retained segment. The first step uses zero differences. The implementation clips elapsed hours to $[0, 168]$ and distances in kilometers to $[0, 50]$, and embeds them as

$$\boldsymbol{e}^\Delta_t = \tanh\left(W_\Delta \begin{bmatrix} \mathrm{clip}(\Delta\tau_t, 0, 168) \\ \mathrm{clip}(\Delta\ell_t, 0, 50) \end{bmatrix} + \boldsymbol{b}_\Delta\right). \tag{1}$$

The per-step feature vector is

$$\boldsymbol{x}_t = \boldsymbol{e}^v_{v_t} \,\|\, \boldsymbol{e}^c_{c_{v_t}} \,\|\, \boldsymbol{e}^a_{a_t} \,\|\, \boldsymbol{e}^w_{w_t} \,\|\, \boldsymbol{e}^r_{r(v_t)} \,\|\, \boldsymbol{e}^\Delta_t, \tag{2}$$

where the POI embedding is in $\mathbb{R}^{d_e}$ and each remaining embedding is in $\mathbb{R}^{d_c}$. Let $\boldsymbol{e}^u_u \in \mathbb{R}^{d_c}$ be a learned user embedding. We concatenate it after feature dropout and use a one-layer, unidirectional LSTM:

$$\boldsymbol{h}_t = \mathrm{LayerNorm}\Big(\mathrm{LSTM}_t\big([\mathrm{Dropout}(\boldsymbol{x}_j) \,\|\, \boldsymbol{e}^u_u]_{j=1}^t\big)\Big). \tag{3}$$

Packed sequences exclude padded positions from recurrent state updates. The vector $\boldsymbol{h}_t \in \mathbb{R}^d$ depends only on the current prefix; each trajectory is encoded independently. There is no bidirectional encoder or access to later positions when predicting the next POI.

Prefix–target alignment and masking. For a retained segment $(v_1, \ldots, v_L)$, position t receives the context through v_t and is supervised by v_{t+1}. Training uses every valid shift $t \in \{1, \ldots, L-1\}$; validation and test read only the state paired with the designated final target. Padding after each true sequence length is excluded by packing and loss masks. If a history exceeds 50 visits, we retain its most recent continuous suffix without changing the original final

target. This keeps target alignment and the causal score definition unchanged across splits.

3.3 Prefix-Dependent Dual Soft Clustering

We maintain two trainable dictionaries:

$$P = \begin{bmatrix} \boldsymbol{p}_1^\top \\ \vdots \\ \boldsymbol{p}_{K_p}^\top \end{bmatrix} \in \mathbb{R}^{K_p \times d}, \qquad S = \begin{bmatrix} \boldsymbol{s}_1^\top \\ \vdots \\ \boldsymbol{s}_{K_s}^\top \end{bmatrix} \in \mathbb{R}^{K_s \times d}. \tag{4}$$

The first supports reusable trajectory-pattern components. The second supports spatially conditioned components. Rather than assigning an entire user or trajectory to a single cluster, we predict a distribution at every prefix:

$$\boldsymbol{\alpha}_t = \mathrm{softmax}\Big((W_\alpha \boldsymbol{h}_t + \boldsymbol{b}_\alpha)/\sqrt{d}\Big), \tag{5}$$

$$\boldsymbol{\beta}_t = \mathrm{softmax}\left(\frac{W_\beta(\boldsymbol{h}_t \,\|\, \boldsymbol{e}^r_{r(v_t)}) + \boldsymbol{b}_\beta}{\sqrt{d + d_c}}\right). \tag{6}$$

The resulting mixtures are

$$\bar{\boldsymbol{p}}_t = P^\top \boldsymbol{\alpha}_t, \qquad \bar{\boldsymbol{s}}_t = S^\top \boldsymbol{\beta}_t. \tag{7}$$

Soft assignments permit different prefix contexts to combine the same components in different proportions. The prototype vectors and assignment networks are optimized jointly through next-POI supervision; assignments are not nearest-centroid labels or the output of a separate trajectory KMeans procedure.

The two kinds of spatial information serve different roles. $r(v_t)$ is a fixed geographic cluster of the observed POI. In contrast, $\boldsymbol{\beta}_t$ indexes a learned K_s-component dictionary whose vectors are not forced to equal geographic centroids. Region context affects the latter's assignment network, but there is no one-to-one mapping between its prototypes and the R partitions. Likewise, since $\boldsymbol{h}_t$ already incorporates the user and POI, the pattern assignments are not identity-free. The architecture shares its dictionaries across users without claiming statistically independent or semantically identified factors.

Differentiable routing and parameter sharing. The matrices P and S are global trainable parameters, but their mixture weights are recomputed for every prefix. Since both retrievals are weighted sums, recommendation gradients jointly update the routers, prototypes, and causal encoder. Training uses neither an expectation–maximization phase nor a hard reassignment or centroid-refresh step. The divisors in Equations (5)–(6) scale router logits according to their input dimensions; they do not introduce a separately tuned temperature. No entropy or load-balancing objective forces equal prototype use. Hence K_p and K_s specify representational capacities, not a guaranteed number of occupied or semantically distinct clusters. Prototype indices are also permutation-symmetric: exchanging dictionary rows together with the corresponding router outputs leaves the represented function unchanged. Their interpretation must therefore rely on learned behavior rather than index identity.

3.4 Factorized Pattern–Space Composition

Instead of adding two unrelated mixture vectors, we transform and interact them:

$$\boldsymbol{g}_t = \tanh(W_m[(A\bar{\boldsymbol{p}}_t) \odot (B\bar{\boldsymbol{s}}_t)] + \boldsymbol{b}_m), \tag{8}$$

where $A, B, W_m \in \mathbb{R}^{d\times d}$, A and B have no bias, and $\odot$ denotes elementwise multiplication. The spatial mixture therefore changes how the pattern mixture contributes, rather than merely occupying additional input coordinates.

Implicit pairwise interaction. The representation before W_m admits the exact expansion

$$(A\bar{\boldsymbol{p}}_t) \odot (B\bar{\boldsymbol{s}}_t) = \sum_{k=1}^{K_p}\sum_{l=1}^{K_s} \alpha_{t,k}\beta_{t,l}[(A\boldsymbol{p}_k) \odot (B\boldsymbol{s}_l)]. \tag{9}$$

Thus, two soft assignments yield an implicit mixture over all pattern–space pairs. The model need not store one unrestricted vector per pair or instantiate a $K_p \times K_s$ tensor at each step. The separable weights $\alpha_{t,k}\beta_{t,l}$ impose a structural constraint: arbitrary pair-specific attention is not available. This is a deliberate compositional bias, not a claim of greater expressiveness than every unfactorized alternative.

Spatially conditioned transformation. For fixed $\bar{\boldsymbol{s}}_t$, Equation (8) can also be written as

$$\boldsymbol{g}_t = \tanh\left(\underbrace{W_m \,\mathrm{diag}(B\bar{\boldsymbol{s}}_t)A}_{T(\bar{\boldsymbol{s}}_t)}\,\bar{\boldsymbol{p}}_t + \boldsymbol{b}_m\right). \tag{10}$$

This makes precise the intuition that a reusable pattern is expressed differently under different spatial contexts. The effective transformation changes with the current spatial mixture. $T(\bar{\boldsymbol{s}}_t)$ is not constrained to be orthogonal, idempotent, or low rank. Accordingly, the projection interpretation is a conditional feature transformation, not a geometric projection theorem.

3.5 User-Specific Adaptation and Gated Fusion

Modulating the shared interaction. We use feature-wise affine conditioning [6] to personalize $\boldsymbol{g}_t$:

$$\begin{aligned} \boldsymbol{\gamma}_u &= \mathrm{sigmoid}(W_\gamma \boldsymbol{e}^u_u + \boldsymbol{b}_\gamma), \\ \boldsymbol{\eta}_u &= W_\eta \boldsymbol{e}^u_u + \boldsymbol{b}_\eta, \\ \tilde{\boldsymbol{g}}_{u,t} &= \boldsymbol{\gamma}_u \odot \boldsymbol{g}_t + \boldsymbol{\eta}_u. \end{aligned} \tag{11}$$

The user-dependent scale attenuates individual dimensions, while the shift changes their offsets. Although these affine parameters are constant for a user, the representation they modulate varies with the trajectory prefix.

A separate personalized branch. To avoid requiring all preference information to pass through the dictionaries, we compute

$$\boldsymbol{q}_{u,t} = \tanh\big(W_q(\boldsymbol{h}_t \,\|\, \boldsymbol{e}^u_u) + \boldsymbol{b}_q\big). \tag{12}$$

This branch has its own transformation, while sharing the causal encoder with the dictionary branch. Its role is a user-conditioned residual pathway, not an additional independently trained user-specific LSTM.

Additive versus projective composition. An additive rule retains contributions independently, whereas a multiplicative interaction lets one branch condition the other. We expose both choices:

$$z^{\text{add}}_{u,t} = \tilde{g}_{u,t} + q_{u,t},$$
$$z^{\text{proj}}_{u,t} = W_\pi(\tilde{g}_{u,t} \odot q_{u,t}) + b_\pi, \tag{13}$$
$$\rho_{u,t} = \text{sigmoid}\big(w_2^\top \text{GELU}(W_1[\tilde{g}_{u,t} \,\|\, q_{u,t}] + b_1) + b_2\big),$$
$$z_{u,t} = \text{Dropout}\Big(\rho_{u,t} z^{\text{add}}_{u,t} + (1-\rho_{u,t}) z^{\text{proj}}_{u,t}\Big). \tag{14}$$

Here $\rho_{u,t} \in (0,1)$ is a *scalar* for each timestep, unlike the feature-wise vector γ_u. The projective path is a learned affine transform of an elementwise product; it does not use the region-indexed low-rank projection module present in other models of the codebase. Both branches are always evaluated, and the gate interpolates their outputs rather than performing discrete expert selection.

Representation-space fusion. Fusion occurs before candidate scoring. The additive path preserves evidence that either the shared or specific branch can contribute independently, whereas the projective path can amplify or suppress coordinates through their elementwise agreement. The scalar gate is conditioned jointly on both branches and can vary across prefixes, but it is shared across representation dimensions and candidates at a given step. Consequently, it cannot perform candidate-specific reranking or sparse expert selection, and both paths always receive gradients. Because the gate is evaluated at every valid prefix, the same user may emphasize additive evidence in one context and projective interaction in another; it is a trajectory-state decision rather than a static user-level coefficient. The gate changes only the contextual preference vector used by the first term of Equation (15); the first-order transition table and candidate bias remain separate. This separation makes the gate interpretable as choosing how to form contextual preference rather than how much transition evidence to inject.

3.6 Full-Candidate Scoring and Optimization

For each candidate $v \in \mathcal{V}$, the model computes

$$s_t(v) = z_{u,t}^\top o_v + a_{v_t}^\top b_v + b_v^{\text{out}}, \tag{15}$$

where $o_v \in \mathbb{R}^d$ is a candidate preference embedding and $a_{v_t}, b_v \in \mathbb{R}^{d_e}$ are separate source and destination transition embeddings. The first term captures the composed contextual preference; the second is a learned first-order transition; the final term is a POI bias. These tables are not tied to the encoder's POI embeddings. The transition term is trained end-to-end, not populated from held-out transition counts.

We normalize over the entire training vocabulary:

$$p_\theta(v \mid u, H_{u,t}) = \frac{\exp(s_t(v))}{\sum_{j\in\mathcal{V}} \exp(s_t(j))}. \tag{16}$$

For a minibatch's valid supervised positions $\mathcal{I}$, the loss is

$$\mathcal{L}_{\text{next}} = -\frac{1}{|\mathcal{I}|} \sum_{(u,t)\in\mathcal{I}} \log p_\theta(v_{t+1} \mid u, H_{u,t}). \tag{17}$$

All valid next-step positions in retained training segments receive supervision, while padding is ignored. At evaluation only the designated final target is scored. The model uses this recommendation loss with optimizer weight decay; it adds no category/region prediction, contrastive, reconstruction, or cluster-balancing loss. Parameter sharing and factorization are therefore inductive biases; they do not prevent cluster collapse.

Table 1: Training-derived vocabularies and eligible sequences under the strict protocol. Training sequences supervise shifted next steps; each validation/test sequence contributes one original final query.

Item	NYC	TKY
Training users	1,047	2,262
Training POI candidates	4,980	7,793
Training categories	318	289
Training sequences	11,022	35,166
Validation queries	1,355	4,770
Test queries	1,300	4,711

Computational properties. At each timestep, assignments and dictionary mixtures require $O(dK_p + (d+d_c)K_s)$ operations up to constant factors, and dense transformations require $O(d^2 + dd_c)$. The dictionary storage is $O((K_p + K_s)d)$, rather than an unrestricted $O(K_pK_sd)$ pair dictionary. The complete output head costs $O(|\mathcal{V}|(d+d_e))$ per timestep and dominates as the candidate catalog grows. These are properties of the computation, not measured speed or memory improvements over baselines. No extra graph propagation, language-model inference, or candidate pruning is required by DualClusterNet.

4 Experiments

We evaluate whether DualClusterNet is competitive with established next-POI model families under a common closed-set protocol, followed by controlled structural and sensitivity analyses on the same NYC and TKY partitions and dataset/query scales used in the main comparison.

4.1 Datasets and Strict Evaluation Protocol

We use the chronological NYC and TKY check-in partitions associated with GETNext [13]. Table 1 reports training-derived vocabulary sizes and eligible queries after applying the same filtering to every method. User, POI, category, coordinate, geographic partition, and transition mappings are fitted on training observations only. All methods rank the same complete training POI vocabulary, including previously visited POIs.

For each held-out trajectory, an unknown POI breaks the known segment; observations across it are never joined. We retain the continuous known suffix ending at the original final check-in, and use one query only when its user and final target occur in the training vocabulary and at least one history item remains. Thus, an unknown final target is rejected rather than replaced by an earlier known POI. Inputs contain observed check-ins only: future time, category, coordinates, and the target POI are never provided to the predictor. The reference configuration retains at most 50 historical visits.

Table 2: Full-candidate test results under the unified training-only protocol (seed 42). Higher is better. Bold: best; underline: second best.

	NYC				TKY			
Method	Acc@1	Acc@5	Acc@10	MRR	Acc@1	Acc@5	Acc@10	MRR
FPMC [8]	0.1700	0.4369	0.5438	0.2917	0.0951	0.2585	0.3600	0.1790
ST-RNN [3]	0.2054	0.4862	**0.5923**	0.3352	0.2309	0.4698	0.5636	0.3408
STGN [14]	0.2115	0.4838	0.5877	0.3361	0.2437	0.4787	0.5602	0.3520
STGCN [14]	0.2162	0.4777	0.5662	0.3334	0.2428	0.4751	0.5621	0.3499
PLSPL [10]	0.2269	0.4777	0.5446	0.3361	0.2371	0.4458	0.5256	0.3365
STAN [5]	0.1023	0.2731	0.3715	0.1866	0.1104	0.2885	0.3755	0.1985
STHGCN [11]	0.2169	**0.4985**	0.5885	0.3411	0.2520	0.4774	0.5665	0.3556
DCHL [2]	0.2377	0.4900	0.5785	0.3509	0.2520	**0.4878**	**0.5699**	0.3606
DualClusterNet (ours)	**0.2400**	0.4969	0.5908	**0.3516**	**0.2664**	0.4844	0.5676	**0.3679**

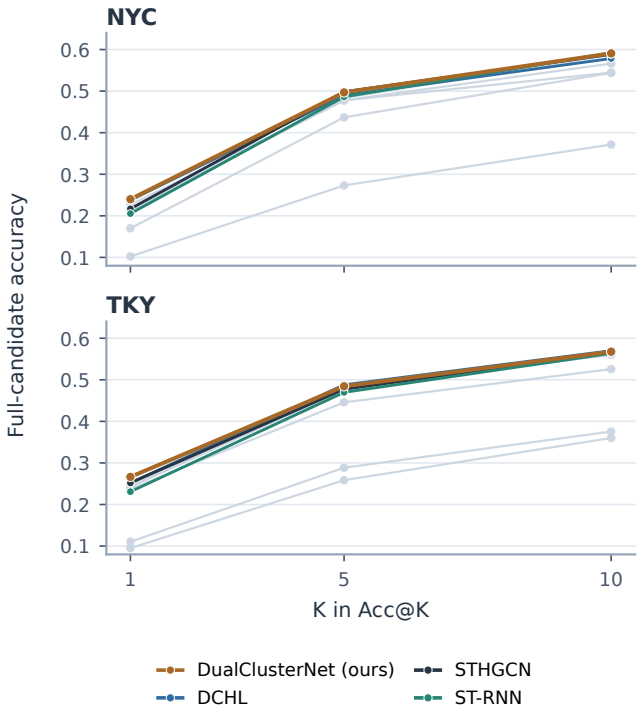


Figure 2: Full-candidate Acc@K for the methods in Table 2. Colored lines highlight DualClusterNet and its strongest competitors; gray lines show the other methods.

4.2 Experiment Setting

We compare DualClusterNet with eight representative methods: FPMC, ST-RNN, STGN, STGCN, PLSPL, STAN, STHGCN, and DCHL. Every implementation uses the same train-only vocabulary, eligible queries, complete candidate set, and validation rule. STGCN denotes the coupled-gate variant introduced with STGN, and STAN uses the latest observed time to respect the query information boundary.

We report Acc@K for $K \in \{1, 5, 10\}$ and MRR. Since each query has one target, Acc@K equals Recall@K under this protocol. The reported runs use seed 42, and checkpoints are selected only on validation data. Metric definitions, model-specific inputs and objectives, architecture settings, parameter counts, and optimization details are provided in Appendix A.

4.3 Main Results

On NYC, DualClusterNet records the highest Acc@1 (0.2400) and MRR (0.3516), narrowly ahead of DCHL (0.2377 and 0.3509). Results at larger K are mixed: STHGCN leads Acc@5 and ST-RNN leads Acc@10. The advantage is therefore concentrated in first-choice ordering and reciprocal rank rather than uniform top-K inclusion.

On TKY, the model obtains the highest completed-run Acc@1 (0.2664) and MRR (0.3679) among the displayed baselines. DCHL is stronger on Acc@5 (0.4878) and Acc@10 (0.5699), so the advantage is again concentrated in first-choice ordering and reciprocal rank. Figure 2 visualizes this distribution of strengths across K.

These results support competitiveness under the unified local protocol, with the strongest evidence on NYC top-one prediction. They do not establish uniform superiority, statistical significance, or that the prototype dictionaries caused the observed margins. The preference-plus-transition head and shared encoder are common sources of predictive signal, which motivates the separate diagnostic checks below.

Metric-level interpretation. The metric pattern distinguishes placing the correct POI first from retaining it in a broader list. On NYC, the Acc@1 margin over DCHL is 0.0023 and the MRR margin is 0.0007; STHGCN leads Acc@5 and ST-RNN leads Acc@10. On TKY, DualClusterNet leads Acc@1 and MRR, but DCHL is higher on Acc@5 and Acc@10. The absolute differences are small, so the table profiles ranking behavior rather than showing that one method dominates every retrieval objective.

The complete-candidate protocol also matters. Every method scores the same training-derived POI vocabulary, including previously visited locations; no method receives a recalled subset. Different filtering, future-time input, or candidate sampling would change the ranking denominator and top-K difficulty. The comparison therefore concerns these unified local implementations and should not be transferred to published numbers from another candidate set or query policy.

Table 3: Controlled ablations on NYC and TKY under the main-study data scale and protocol (Acc@1/MRR). The controlled-study seed differs from the seed-42 main run and is fixed across variants.

Variant	NYC		TKY	
	Acc@1	MRR	Acc@1	MRR
Full	**0.3007**	**0.4209**	**0.3403**	**0.4507**
Backbone	0.2713	0.3926	0.3147	0.4263
w/o pattern	0.2837	0.4058	0.3269	0.4376
w/o spatio	0.2864	0.4083	0.3291	0.4405
Concat pattern–space	0.2926	0.4147	0.3338	0.4449
w/o FiLM	0.2971	0.4192	0.3412	0.4513
w/o specific	0.2819	0.4027	0.3246	0.4358
Add-only	0.2958	0.4176	0.3374	0.4472
Projection-only	0.2893	0.4104	0.3327	0.4426
Fixed gate $\rho = 0.5$	0.2984	0.4195	0.3396	0.4493

4.4 Ablation Study

The controlled analyses use the same NYC and TKY data versions, split scales, training-only vocabularies, filtering rules, and complete candidate sets as the main comparison. Within the ablation study, all variants share the same encoder, prediction head, optimizer, training budget, validation rule, and random seed. The Full control uses the main model and hyperparameter configuration.

Each variant targets one representation-side decision while retaining full-candidate scoring. Backbone bypasses the dual dictionaries and personalized composition but keeps the causal trajectory encoder and output head. The w/o pattern and w/o spatio variants retain only the complementary dictionary contribution. Concat pattern–space replaces the factorized product with a learned concatenation map, directly testing whether multiplicative composition is preferable to a generic fusion of both mixtures. The w/o FiLM and w/o specific variants isolate the two personalization mechanisms. Add-only, Projection-only, and Fixed gate $\rho = 0.5$ then distinguish the two fusion paths from the benefit of adapting their balance to each prefix. All variants retain the same transition term, so their differences concern contextual representation rather than candidate-set construction.

Table 3 shows the ablation pattern. Removing the complete compositional stack (Backbone) lowers Acc@1 by 2.94 and 2.56 percentage points on NYC and TKY, respectively, while removing the specific branch lowers it by 1.88 and 1.57 points. The pattern and spatial branches and the product interaction each provide smaller positive differences. The FiLM and fixed-gate effects are small and dataset-dependent; on TKY, removing FiLM is slightly higher than Full. These results identify the full stack and specific branch as the largest contributors in the reported runs, while the adaptive gate provides a modest additional gain.

The ablation gaps are consistent across the two cities. Removing the backbone lowers Acc@1 by 0.0294 on NYC and 0.0256 on TKY; removing the specific branch lowers it by 0.0188 and 0.0157, respectively. Replacing the pattern–space product by a learned concatenation is less damaging, with drops of 0.0081 and 0.0065, while the additive-only and projection-only personalized-fusion variants both remain below Full. The learned gate is slightly better than a fixed $\rho = 0.5$ in both cities. FiLM is the exception to a monotone interpretation: removing it raises TKY Acc@1 by 0.0009 and MRR by 0.0006, so its contribution is dataset-dependent rather than uniformly positive. Every row retains the same first-order transition term; this comparison isolates representation-side changes but does not estimate the transition term's independent contribution.

4.5 Parameter Sensitivity

We vary K_p, K_s, and R one at a time while retaining the Full architecture, data protocol, training budget, and all non-scanned hyperparameters. The reference point in each sweep therefore matches the main model configuration apart from the controlled-study random seed.

The three axes govern different capacities. K_p controls the number of reusable trajectory-pattern vectors, whereas K_s controls the number of learned spatial response vectors available to the prefix router. By contrast, R sets the resolution of the fixed training-coordinate partition supplied as observed context; it does not change the size of the learned spatial dictionary. A small value can merge heterogeneous behavior into too few components, while a large value can disperse supervision across weakly used components or overly fine regions. We select settings by validation Acc@1 and vary one axis at a time, so the curves measure local sensitivity around the reference configuration rather than all interactions among the three capacities.

The one-at-a-time sweeps in Table 4 and Figure 3 show interior optima: $K_p = 64$ is highest on both cities, whereas the preferred K_s and R differ by city. NYC favors $K_s = 32$ and $R = 128$, while TKY favors $K_s = 64$ and $R = 256$; the largest tested region count lowers validation accuracy on both datasets. This pattern supports tuning the two dictionary capacities and the geographic partition separately rather than treating them as one shared scale.

4.6 Case Study

The case diagnostics in Table 5 include a Full correction, a tie, a regression, and a joint failure. Each displayed rank is computed against the corresponding city's complete training-derived POI vocabulary, exactly as in the main evaluation; no 100-candidate sample or target-informed shortlist is used. Full and Backbone are each top-1 correct in two of four cases, so the cases complement the aggregate metrics by exposing both gains and failure modes.

The four outcomes show why aggregate gains should not be interpreted as uniform per-query improvements. In C1, Full moves the target from rank 7 to rank 1, with $\rho = 0.7213$ assigning more weight to the additive path. C2 is already solved by Backbone and remains rank 1 under Full, indicating that the compositional module need not disturb an easy sequential prediction. C3 is the reverse: Backbone ranks the target first, while Full lowers it to rank 9 and places more weight on the projective path through $\rho = 0.3129$. In C4, neither representation places the target in the top 10. The symbolic paths anonymize the underlying check-ins, and the gate

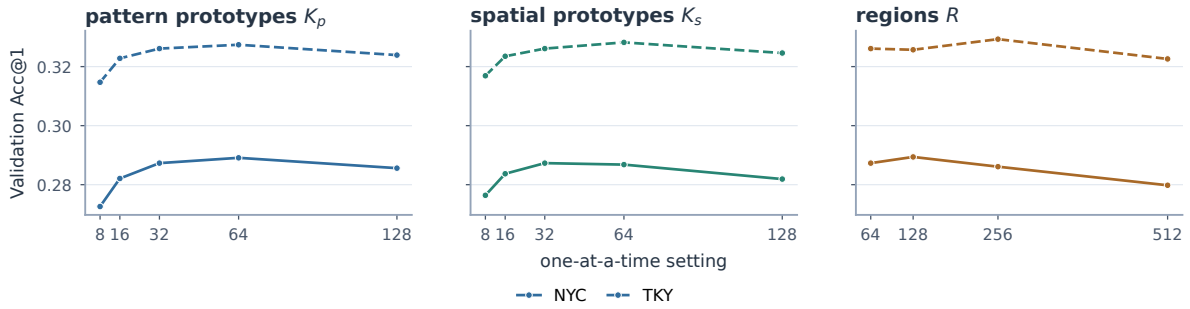


Figure 3: One-at-a-time validation sweeps on the main-study datasets. Solid lines denote NYC and dashed lines TKY.

Table 4: One-parameter validation sweeps on NYC and TKY. † marks the main-model setting under the controlled-study seed; each other row changes only the named parameter. Bold marks the per-city maximum.

Sweep	Setting	NYC	TKY
Pattern K_p	8	0.2726	0.3147
	16	0.2821	0.3228
	32†	0.2873	0.3261
	64	**0.2891**	**0.3274**
	128	0.2856	0.3239
Spatial K_s	8	0.2764	0.3169
	16	0.2837	0.3235
	32†	**0.2873**	0.3261
	64	0.2868	**0.3282**
	128	0.2819	0.3246
Regions R	64†	0.2873	0.3261
	128	**0.2894**	0.3257
	256	0.2861	**0.3293**
	512	0.2798	0.3226

Table 5: Case diagnostics ranked over the corresponding city's complete training-derived POI vocabulary (lower is better); no sampled candidate pool is used.

Case	Path	Full	Back.	ρ	Outcome
C1	A–B–C→D	1	7	0.7213	Full only
C2	E–F–E→G	1	1	0.4387	Both correct
C3	H–I–J→K	9	1	0.3129	Backbone only
C4	L–M–N→O	18	12	0.5816	Both fail

values are descriptive model states rather than causal explanations of each outcome.

Overall discussion. The results indicate that DualClusterNet is most useful when the top-ranked choice depends jointly on trajectory state and local context. The pattern dictionary reuses sequential regularities across users, the spatial dictionary changes how those regularities are expressed nearby, and the specific branch preserves user evidence when the current prefix does not match a shared component. This explains why Acc@1 and MRR can improve while a competing model retains more alternatives at Acc@5 or Acc@10. The controlled gaps suggest a practical order for deployment: preserve the shared backbone and specific branch first, then tune K_p, K_s, and R under a fixed latency budget. The full-candidate protocol should remain unchanged when comparing such variants, because candidate recall and ranking quality otherwise become confounded.

Component comparisons should keep the encoder, output head, data filtering, and checkpoint rule fixed. Capacity sweeps should vary one dictionary or region count at a time, and repeated seeds should be paired at query level to quantify the small margins. If approximate retrieval is introduced later, its recall of the exact full-candidate target should be reported before end-to-end latency. These checks extend the present evidence without treating tuning or retrieval engineering as an architectural gain.

Limitations. The city comparison and each controlled configuration currently use one seed. The closed-set evaluation covers known users and POIs only; it does not test cold-start users, unseen locations, catalog growth, or cross-city transfer. The dictionaries have no semantic supervision, so prototype interpretability, assignment entropy, and geographic coherence remain open questions. Repeated paired runs are needed to quantify uncertainty around the small margins and dataset-dependent ablations, and a matched removal of the transition score is needed to isolate its contribution.

5 Conclusion

We presented DualClusterNet, which combines prefix-conditioned pattern and spatial mixtures, user adaptation, and additive–projective fusion with contextual and transition scoring. Under the training-only, original-final-target protocol, the available single run leads Acc@1 and MRR on NYC and remains competitive on TKY; the evidence supports competitiveness rather than universal superiority. On the same data scale, Full exceeds the backbone and concatenation controls in both cities, FiLM remains dataset-dependent, and sensitivity sweeps favor city-specific capacities. Stronger attribution requires repeated seeds, a transition-score control, cluster diagnostics, and efficiency measurements. Future work should also test unseen users and POIs under the same information boundary.

References

[1] Jie Feng, Yong Li, Chao Zhang, Funing Sun, Fanchao Meng, Ang Guo, and Depeng Jin. 2018. DeepMove: Predicting Human Mobility with Attentional Recurrent Networks. In *Proceedings of the 2018 World Wide Web Conference (WWW '18)*. Association for Computing Machinery, New York, NY, USA, 1459–1468. doi:10.1145/3178876.3186058

[2] Yantong Lai, Yijun Su, Lingwei Wei, Tianqi He, Haitao Wang, Gaode Chen, Daren Zha, Qiang Liu, and Xingxing Wang. 2024. Disentangled Contrastive Hypergraph Learning for Next POI Recommendation. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '24)*. Association for Computing Machinery, New York, NY, USA, 1452–1462. doi:10.1145/3626772.3657726

[3] Qiang Liu, Shu Wu, Liang Wang, and Tieniu Tan. 2016. Predicting the Next Location: A Recurrent Model with Spatial and Temporal Contexts. *Proceedings of the AAAI Conference on Artificial Intelligence* 30, 1 (2016), 194–200. doi:10.1609/aaai.v30i1.9971

[4] Yuwen Liu, Lianyong Qi, Xingyuan Mao, Weiming Liu, Fan Wang, Xiaolong Xu, Xuyun Zhang, Wanchun Dou, Xiaokang Zhou, and Amin Beheshti. 2025. Hyperbolic Variational Graph Auto-Encoder for Next POI Recommendation. In *Proceedings of the ACM Web Conference 2025 (WWW '25)*. Association for Computing Machinery, New York, NY, USA, 3267–3275. doi:10.1145/3696410.3714804

[5] Yingtao Luo, Qiang Liu, and Zhaocheng Liu. 2021. STAN: Spatio-Temporal Attention Network for Next Location Recommendation. In *Proceedings of the Web Conference 2021 (WWW '21)*. Association for Computing Machinery, New York, NY, USA, 2177–2185. doi:10.1145/3442381.3449998

[6] Ethan Perez, Florian Strub, Harm de Vries, Vincent Dumoulin, and Aaron Courville. 2018. FiLM: Visual Reasoning with a General Conditioning Layer. *Proceedings of the AAAI Conference on Artificial Intelligence* 32, 1 (2018), 3942–3951. doi:10.1609/aaai.v32i1.11671

[7] Xuan Rao, Shuo Shang, Lisi Chen, Renhe Jiang, and Peng Han. 2025. Disentangled and Personalized Representation Learning for Next Point-of-Interest Recommendation. In *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence*. International Joint Conferences on Artificial Intelligence Organization, 7697–7705. doi:10.24963/ijcai.2025/856

[8] Steffen Rendle, Christoph Freudenthaler, and Lars Schmidt-Thieme. 2010. Factorizing Personalized Markov Chains for Next-Basket Recommendation. In *Proceedings of the 19th International Conference on World Wide Web (WWW '10)*. Association for Computing Machinery, New York, NY, USA, 811–820. doi:10.1145/1772690.1772773

[9] Ke Sun, Tieyun Qian, Tong Chen, Yile Liang, Quoc Viet Hung Nguyen, and Hongzhi Yin. 2020. Where to Go Next: Modeling Long- and Short-Term User Preferences for Point-of-Interest Recommendation. *Proceedings of the AAAI Conference on Artificial Intelligence* 34, 1 (2020), 214–221. doi:10.1609/aaai.v34i01.5353

[10] Yuxia Wu, Ke Li, Guoshuai Zhao, and Xueming Qian. 2022. Personalized Long- and Short-term Preference Learning for Next POI Recommendation. *IEEE Transactions on Knowledge and Data Engineering* 34, 4 (2022), 1944–1957. doi:10.1109/TKDE.2020.3002531

[11] Xiaodong Yan, Tengwei Song, Yifeng Jiao, Jianshan He, Jiaotuan Wang, Ruopeng Li, and Wei Chu. 2023. Spatio-Temporal Hypergraph Learning for Next POI Recommendation. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '23)*. Association for Computing Machinery, New York, NY, USA, 403–412. doi:10.1145/3539618.3591770

[12] Dingqi Yang, Benjamin Fankhauser, Paolo Rosso, and Philippe Cudre-Mauroux. 2020. Location Prediction over Sparse User Mobility Traces Using RNNs: Flashback in Hidden States!. In *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence*. International Joint Conferences on Artificial Intelligence Organization, 2184–2190. doi:10.24963/ijcai.2020/302

[13] Song Yang, Jiamou Liu, and Kaiqi Zhao. 2022. GETNext: Trajectory Flow Map Enhanced Transformer for Next POI Recommendation. In *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '22)*. Association for Computing Machinery, New York, NY, USA, 1144–1153. doi:10.1145/3477495.3531983

[14] Pengpeng Zhao, Haifeng Zhu, Yanchi Liu, Jiajie Xu, Zhixu Li, Fuzhen Zhuang, Victor S. Sheng, and Xiaofang Zhou. 2019. Where to Go Next: A Spatio-Temporal Gated Network for Next POI Recommendation. *Proceedings of the AAAI Conference on Artificial Intelligence* 33, 1 (2019), 5877–5884. doi:10.1609/aaai.v33i01.33015877

[15] Hongli Zhou, Zhihao Jia, Haiyang Zhu, and Zhizheng Zhang. 2024. CLLP: Contrastive Learning Framework Based on Latent Preferences for Next POI Recommendation. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '24)*. Association for Computing Machinery, New York, NY, USA, 1473–1482. doi:10.1145/3626772.3657730

A Implementation and Reproducibility Details

A.1 Shared Data, Training, and Selection

All compared methods are implemented as PyTorch modules within one evaluation framework. Their active entry points follow `baselines/name/run_name.py`; model construction is in the corresponding `train.py` and `model.py`. Every entry point delegates data handling, optimization, validation, and testing to the same training engine. The method-specific inputs, retained mechanisms, objectives, and protocol adaptations are documented below.

The engine constructs user, POI, category, coordinate, graph, hypergraph, and clustering state from the training split only. A model's forward pass receives only observed user and prefix fields: POI, category, hour, weekday, elapsed time, traveled distance, normalized observed time, and valid length, with each method using the subset listed below. It never receives the target POI, target category, or target time. Every method returns a score tensor over the complete training POI vocabulary at every valid prefix. Training uses every shifted next-step target in a retained training segment; validation and test use one original final target per eligible trajectory. Evaluation is always full-candidate cross-entropy and exact full-vocabulary ranking, irrespective of the training objective.

All runs in Table 2 use seed 42 and a maximum history length of 50. Training and evaluation batch sizes are 32. Optimization uses Adam, learning rate 10^{-3}, weight decay 10^{-5}, gradient clipping at 5, and at most 30 epochs. A plateau scheduler halves the learning rate after three epochs without lower validation loss. The validation checkpoint maximizes 4 Acc@1+Acc@10; training stops after eight epochs without improvement in that score. Table 7 lists the fixed architecture configuration used for each method.

A.2 Metrics and Reference Configuration

For a query with ground-truth rank r_q in the complete candidate list, the reported metrics are

$$\text{Acc@}K = \frac{1}{|Q|}\sum_{q\in Q}\mathbf{1}[r_q \le K], \qquad \text{MRR} = \frac{1}{|Q|}\sum_{q\in Q}\frac{1}{r_q}. \tag{18}$$

We use $K \in \{1, 5, 10\}$, resolve ties by candidate-index order, and report query-weighted means. With one target per query, Acc@K is numerically identical to Recall@K.

The reference DualClusterNet configuration uses $d_e = 64$, $d_c = 32$, $d = 128$, $K_p = K_s = 32$, $R = 64$, and dropout 0.2. Geographic KMeans is fitted on training coordinates with ten initializations, at most 300 iterations, and seed 42. The same optimizer, training budget, validation selection rule, and full-candidate evaluation are used for the controlled variants unless the named sensitivity parameter is changed.

A.3 Method-Specific Implementation Notes

Sequence and transition baselines. FPMC retains long-term user-preference and first-order Markov factors, with a basket size of one. ST-RNN uses learned transition matrices interpolated from observed time and distance gaps. STGN implements separate short- and long-interval gates; STGCN couples its input and forgetting behavior in the local port. All four output exact scores over the shared POI vocabulary, so no sampled candidate list changes the test task.

Table 6: Local baseline mechanisms and protocol adaptations. "Observed gaps" are computed only between events in the available prefix. All scoring is over the complete training POI vocabulary.

Method	Retained mechanism	Model-visible inputs	Local adaptation and training objective
FPMC	User–candidate preference plus first-order previous–next POI factorization	User and observed POI prefix	Singleton next-POI form; BPR with 10 uniformly sampled negatives per valid training step
ST-RNN	Recurrent transition with linearly interpolated temporal and distance matrices	User, POI, observed elapsed-time and distance gaps	Log-spaced interpolation over 12 time and 12 distance bins; full-vocabulary cross-entropy (CE)
STGN	Separate spatio-temporal recurrent gates	User, POI, observed time and distance gaps	Uses only observed gaps and a user residual at scoring; full CE
STGCN	Parameter-reduced coupled-gate variant introduced with STGN	Same as STGN	This name denotes the coupled STGN cell, not a graph convolution; full CE
PLSPL	POI-short, category-short, and causal long-term branches with a user-specific mixture	User, POI, category, hour, and weekday	Causal prefix attention; no future category or time; full CE
STAN	Two attention stages with learned spatial/temporal interval interpolation	User, POI, hour, weekday, training coordinates, and observed elapsed time	Uses the latest observed time, recomputes each prefix causally, and chunks only computation (not candidates); sampled CE with 10 negatives
STHGCN	Session, user–POI, region, region–time, and transition hypergraph/graph views, then GRU and causal Transformer	User, POI, category, hour, and weekday, plus train-derived structures	Every structural view is reconstructed from train; full CE and no nonzero auxiliary loss
DCHL	Factorized collaborative, geographic, and directed-transition views with adaptive fusion and a causal Transformer	User, POI, category, hour, and weekday, plus train-derived structures	Train-only graphs; full CE plus symmetric InfoNCE (weight 0.05) and factor-decorrelation loss (weight 0.01)

Table 7: Architecture settings used in the reported runs. Parameter counts and validation-selected epochs are shown as NYC/TKY.

Method	Singleton architecture setting	Parameters (NYC/TKY)	Best epoch (NYC/TKY)
FPMC	MF dimension 64; Markov dimension 64; dropout 0	1,028,212 / 1,648,881	19 / 7
ST-RNN	hidden 64; 12 time and 12 distance bins; maxima 720 h and 500 km; dropout 0.2	811,956 / 1,252,593	30 / 29
STGN	POI embedding 64; hidden 64; dropout 0.2	759,540 / 1,200,177	25 / 25
STGCN	POI embedding 64; hidden 64; coupled gates; dropout 0.2	751,284 / 1,191,921	14 / 27
PLSPL	POI embedding 64; context 32; hidden 96; dropout 0.2	1,057,433 / 1,551,923	28 / 28
STAN	embedding 50; candidate-computation chunk 512; prefix checkpointing enabled	317,850 / 519,250	17 / 24
STHGCN	POI embedding 64; context 24; hidden 128; two hypergraph layers; one four-head Transformer layer; 64 regions; dropout 0.2	754,732 / 966,041	5 / 10
DCHL	embedding 64; context 24; four factors; one hypergraph layer; two four-head Transformer layers; 16 geographic neighbors; temperature 0.2; at most 384 contrastive POIs; dropout 0.2	476,536 / 687,845	9 / 17

Preference and attention baselines. PLSPL retains separate short-term POI and category paths and a long-term attention path, with causal attention at each prefix. STAN uses the latest observed timestamp so that every query depends only on its available prefix. Its candidate chunk size controls memory only; all POIs remain in the denominator and ranking.

Graph and hypergraph baselines. STHGCN constructs five train-only structures: session–POI, user–POI, region, region–time, and directed transition. DCHL builds user–POI, geographic-neighbor, and bidirectional-transition views from train and retains its contrastive and disentanglement regularizers. No structural statistic is fitted on validation or test observations.